EntropyChain: Verifiable Randomness as Consensus

Oct 10, 2026 · EntropyChain project

Abstract

EntropyChain is a CPU-mineable proof-of-work blockchain in which Every block publishes a 32-byte random beacon, derived from its proof-of-work hash, that anyone can verify. Mining uses Argon2id, a memory-hard function that keeps commodity CPUs competitive with GPUs and specialized hardware. The seeds form a public randomness beacon that native transactions consume directly: lotteries pick winners and NFT mints derive traits from past blocks, with no oracle and no off-chain trust.

Instead of a general-purpose virtual machine, EntropyChain defines eleven typed transactions covering transfers, fungible tokens, NFTs and lotteries. Every rule is part of consensus, so there are no contracts to audit and no gas to estimate. The native currency, ENTRO, has a fixed maximum supply of 133.7 million, issued as block rewards that halve every 1,337,000 blocks. A public testnet (testnet-4) has been live since 10 October 2026.

1. Motivation

EntropyChain targets three problems that general-purpose blockchains leave open: specialized mining hardware, untrustworthy on-chain randomness, and smart-contract risk.

Mining has left commodity hardware. Fast hash functions such as SHA-256 reward purpose-built ASICs so heavily that a laptop no longer finds blocks at any useful rate. Permissionless mining becomes, in practice, mining by whoever can buy the hardware.

On-chain randomness is awkward. Most chains offer no native source of unpredictable, verifiable randomness. Applications fall back on one of four workarounds, each with a cost:

- Reading a recent block hash, which the block's miner can bias.
- Paying an oracle service, which adds a trusted third party and a fee.
- Running commit-reveal among participants, which needs every participant to stay online and can be stalled by one who does not reveal.

- Running verifiable delay function (VDF) infrastructure, which is still heavy to operate.

General-purpose virtual machines invite contract bugs. Lotteries and NFT mints are usually written as smart contracts. Each new contract is new code that must be audited, and errors such as reentrancy have repeatedly cost users funds.

EntropyChain answers all three in the protocol itself: memory-hard mining that keeps CPUs competitive, a randomness seed in every block, and native transaction types in place of contracts.

2. Design overview

The protocol rests on three principles, and everything else follows from them.

1. **CPU-reachable mining.** A solo miner on a recent laptop should find blocks at a meaningful rate without special hardware.
2. **Randomness as a verifiable primitive.** Anyone should be able to recompute, from public data, that a block's seed was derived correctly.
3. **Trust-minimized native features.** Anything that consumes randomness should depend only on chain state and protocol rules, never on an outside service or quorum.

The chain is a sequence of blocks linked by hash. Each block carries a header, an ordered list of transactions, and the block's entropy seed. Nodes replay the transactions in order to build the shared state: account balances, token registries, NFT ownership and lottery escrows. Nodes gossip blocks and transactions over a websocket peer-to-peer network, and each serves a JSON RPC interface for wallets.

Throughput is deliberately secondary. Blocks target a 30-second interval, the reference miner includes at most 100 transactions per block, and the mempool is capped at 10,000 pending transactions.

3. Proof-of-Entropy consensus

A block is valid when the Argon2id hash of its header and nonce falls below a target set by the block's difficulty; the valid chain with the most cumulative work wins.

3.1 Block header

The header is a fixed 132-byte serialization. The proof-of-work nonce is not part of it; it is appended separately when hashing.

Field	Size (bytes)	Encoding	Meaning
height	4	little-endian uint32	Position in the chain
prev_hash	32	raw	Hash of the parent block
merkle_root	32	raw	Merkle root of the transaction hashes
timestamp	8	little-endian float64	Unix time in seconds
difficulty	8	little-endian uint64	Required difficulty
entropy_seed	32	raw	The block's randomness contribution (section 4)
miner_nonce	16	raw	Miner-chosen input to the seed

The block hash is double SHA-256 of the header followed by the 8-byte little-endian proof-of-work nonce x. Transaction hashes are paired into a Merkle root with double SHA-256; an odd layer repeats its last hash.

3.2 Proof of work

With header bytes h and nonce x, a block is valid when

```
\mathrm{Argon2id}\big(h \,\|\, x,\ \mathrm{salt}=\mathrm{SHA256d}
(h)_{[0..16)},\ t=1,\ m,\ p=1\big) < \left\lfloor \frac{2^{256}}{d}
\right\rfloor
```

where d is the difficulty and the output is 32 bytes read as a big-endian integer. The production memory cost is m = 16,384 KiB (16 MB per attempt). The public testnet runs a relaxed m = 1,024 KiB so that single machines stay practical; the two settings are not consensus-compatible.

Argon2id won the 2015 Password Hashing Competition and is built to resist time-memory trade-offs: an attacker cannot cut the memory each attempt needs by spending more compute. A GPU has thousands of cores but limited fast memory per core, so it cannot run thousands of 16 MB attempts at once. A custom chip gains little unless it also matches commodity DRAM bandwidth.

3.3 Difficulty retargeting

Difficulty starts at 1,000 and stays fixed through block 39. From block 40 on, every 20th block retargets from the time the previous window took:

$$d_{\text{new}} = \operatorname{clamp}\!\left(\left\lfloor \frac{d \cdot 30 \cdot 20}{\max(\Delta t, 1)} \right\rfloor,\ \left\lfloor \frac{d}{4} \right\rfloor,\ 4d \right)$$

Here Δt is the timestamp of the parent minus the timestamp 19 blocks before it, in whole seconds, and the target block time is 30 seconds. The clamp limits any single change to a factor of four in either direction. The first window does not retarget because it would measure from the genesis timestamp, which is fixed in advance rather than mined.

3.4 Fork choice and timestamps

Nodes follow the valid chain with the greatest cumulative work, the sum of its blocks' difficulties. When a heavier branch appears, a node rolls back to the common ancestor, applies the new branch, and returns transactions that only the old branch contained to its mempool.

Two timestamp rules limit manipulation of difficulty and lottery timing:

- A block's timestamp must be later than the median of the previous 11 blocks.
- A newly received block may be at most 2 hours ahead of the receiving node's clock.

4. The randomness beacon

Every block publishes a 32-byte beacon derived from its block hash, so no one, the miner included, can know it before the block is solved.

4.1 Beacon derivation

For block i with block hash h_i:

$$r_i = \mathrm{SHA256}\big(\mathrm{SHA256}(\text{"entropy"} \,\|\, h_i)\big)$$

The block hash covers the whole header and the proof-of-work nonce, so the beacon exists only once a valid proof of work has been found. Lotteries and random NFT mints consume these beacons directly.

4.2 Verifiability

Verification needs only the block hash, which every node already has. There is no beacon operator, no committee and no off-chain computation. Any wallet or explorer can recompute a beacon, a lottery winner or an NFT's traits from the chain alone.

4.3 Bias analysis

A non-miner cannot influence any beacon: they see it only after the block is published. The miner of a block can influence its beacon only by withholding a solved block and mining another, which forfeits the block reward R each time. If a fraction q of possible beacons gives the outcome the miner wants, the expected cost of forcing it is

```
\text{cost} = R \left( \frac{1}{q} - 1 \right)
```

For a lottery with N equally likely entrants, q = 1/N, so a miner gains by biasing the draw only when the prize exceeds R(N − 1). With R = 50 ENTRO and ten entrants, that is 450 ENTRO. High-value lotteries can raise the cost further by aggregating beacons (section 6.1): a miner must then control every block in the window.

4.4 What changed from testnet-3

Testnet-3 drew randomness from a header field, SHA256d(parent hash ‖ miner nonce), whose miner nonce is chosen before the proof of work begins. A miner could try millions of nonces in under a second and mine whichever seed it preferred, choosing lottery winners at almost no cost. Random NFT mints also referenced a block already on chain, so the minter could try token ids until the traits suited them. Testnet-4 fixed both: randomness now comes from the block hash, and random mints draw from a block mined after the mint. The header field remains for format compatibility but is no longer used.

5. Native transactions

EntropyChain has no virtual machine: eleven typed transactions cover everything the chain can do, and each type's rules are part of consensus.

Type	What it does	Key rules
transfer	Moves ENTRO or a token	Amount > 0; sender covers amount, plus fee when in ENTRO

Type	What it does	Key rules
token_create	Registers a fungible token; full supply goes to the creator	Symbol unique, ASCII, not ENTRO; supply > 0; decimals 0–36
token_mint	Issues more of a token	Only the token's creator
nft_create	Registers an NFT collection	Collection id unique
nft_mint	Mints an NFT with caller-supplied traits	Only the collection's creator; token id unused
nft_random_mint	Mints an NFT whose traits a later block's beacon reveals	As nft_mint; names no entropy block (section 6.2)
nft_transfer	Moves an NFT	Only its current owner
lottery_create	Opens a lottery and escrows the prize	Draw height at least 2 blocks ahead; prize > 0; optional draw window of 1–16 blocks
lottery_enter	Buys a ticket	Before the draw block; one ticket per address
lottery_draw	Pays the winner	Only after the whole draw window; at least one entrant
coinbase	Pays the block reward plus fees	First transaction of every block, and only there

5.1 Hashing and signatures

A transaction's hash is double SHA-256 of a canonical JSON encoding of its type, sender, timestamp, nonce, data and fee, with keys sorted. The signature is excluded from the hash, so it cannot be altered to change a transaction's identity.

Transactions are signed with ECDSA over secp256k1, the curve Bitcoin uses, on the SHA-256 of that same encoding. Signatures are 64 bytes (r and s concatenated) with deterministic nonces per RFC 6979. An address is the Base58Check encoding of version byte 0x21 followed by RIPEMD-160 of SHA-256 of the public key, so every address starts with “E”.

5.2 Fees and validation

Fees are integers in the smallest ENTRO unit, never negative, and go to the miner through the coinbase. Each block's coinbase must equal the block reward plus the sum of its fees exactly.

Transactions in a block are validated in order, each against the state the previous ones left. A transaction already on chain is rejected if it appears again, which prevents replay. Amounts must be whole numbers that fit in 64 bits, and all balance arithmetic is checked for overflow.

The mempool holds up to 10,000 pending transactions. When it is full, a new transaction must pay a higher fee than the cheapest one waiting, which it then evicts.

6. Applications

Lotteries and random NFT mints run entirely on chain: the protocol holds the funds, reads the seed and computes the result, so no operator can intervene.

6.1 Lotteries

A lottery moves through three transactions. The organizer's lottery_create escrows the prize and fixes a ticket price, a draw height D and an optional draw window of k blocks (1 to 16, default 1). Anyone may lottery_enter, paying the ticket price in ENTRO, in any block below D. Once block D + k − 1 has been mined, anyone may submit lottery_draw, and the protocol selects the winner from the beacons of blocks D to D + k − 1:

```
R = \begin{cases} r_D & k = 1 \\ \mathrm{SHA256d}(r_D \,\|\, \cdots \,\|\,
r_{D+k-1}) & k > 1 \end{cases} \qquad \text{winner} = \text{entrants}\big[\,
\mathrm{SHA256}(R \,\|\, \text{lottery\_id}) \bmod N \,\big]
```

Entrants are kept in the order their tickets were included, and N is their count. The winner receives the prize plus every ticket paid. Whoever submits the draw has no influence on the result, because the draw window is fixed at creation and entries close before it begins. A window of k blocks means a miner would have to win and bias all k of them. Lotteries in a token other than ENTRO must have a ticket price of 0, so ticket money is never paid out in a different currency.

6.2 Random NFT mints

The nft_random_mint transaction names a collection, a token id and a recipient, but no source of randomness. If the mint is included in block H, its traits come from the beacon of block H + 1, which does not exist when the minter signs; until that block is mined, the NFT reads as pending. The traits are one hash of that beacon, the collection id and the token id:

```
h = \mathrm{SHA256}(r_{H+1} \,\|\, \text{collection\_id} \,\|\,
\text{token\_id})
```

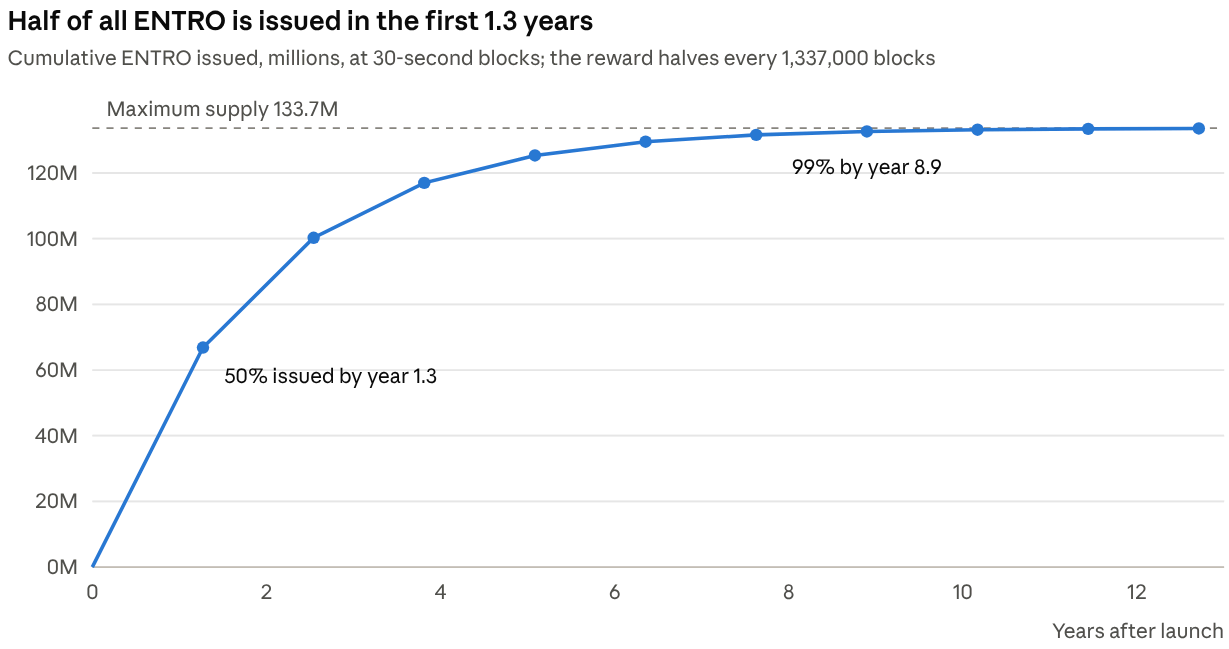
Trait	Derived from	Range
rarity	byte 0 of h	0–255
power	byte 1	0–255
element	byte 2 mod 5	fire, water, earth, air, void
color_seed	bytes 3–5	24-bit colour
is_legendary	byte 6 < 8	true for 1 in 32 (3.125%)

For example, testnet-4 block 50 has beacon d33a4e01…1fecae. A mint of collection “cats”, token “001” revealed by that block gets rarity 150, power 107, element air, colour 2b454d, and is not legendary. Anyone can repeat that calculation from the chain, and because the beacon did not exist when the mint was signed, the minter could not have aimed for it.

7. Monetary policy

ENTRO's supply is capped at 133,699,999.85 coins, issued only as block rewards that halve every 1,337,000 blocks, about every 15 months at 30-second blocks.

- **Units.** One ENTRO is 100,000,000 units; all amounts on chain are whole units.
- **Reward.** The first era pays 50 ENTRO per block. Each halving is an integer shift, so the reward falls to zero after era 33, roughly 42 years in.
- **No premine.** The genesis block's coinbase pays nothing; every ENTRO in existence was mined.
- **Fees.** Fees move existing ENTRO to miners and create none.



Computed from the protocol's reward schedule (params.go) · first 10 of 64 eras

Issuance is front-loaded: the first era mints half of all ENTRO, and 99% exists within nine years.

8. Network and implementation

The reference node is written in Go and ships as a single binary; it replaced the original Python prototype on the public network in October 2026.

8.1 Peer-to-peer protocol

Nodes exchange JSON messages over websockets, one message per frame. A connection opens with a handshake that carries the network id, the sender's tip and its cumulative work; peers on a different network are dropped. New blocks and transactions spread by announcement: a node announces a hash, and peers that lack it request the full object.

A node that falls behind downloads blocks in batches of 100. A block whose parent is unknown is held as an orphan while the node fetches its ancestors, which also resolves deep forks. A mining node waits until it has a peer and is not behind any of them, so it never builds a private chain while still connecting.

Abuse limits: a 1 MB maximum message, at most three connections per IP address, and a misbehaviour score that bans a peer's address for one hour after 100 points, such as four invalid blocks.

8.2 Software

Component	Role	Notes
entropyd	Full node, miner, RPC, block explorer	One ~9 MB binary; crash-safe storage (bbolt)
entropy	Command-line wallet	Keys, transfers, tokens, NFTs, lotteries
EntropyChain Miner	Desktop miner and wallet for Windows	Embeds a full node; one click to mine
Web wallet	Browser wallet at wallet.entropychain.io	Keys stay in the browser, encrypted with a passphrase
Faucet	Free testnet ENTRO at faucet.entropychain.io	100 ENTRO per address per 24 hours
Explorer	explorer.entropychain.io	Blocks, transactions, addresses, tokens, lotteries

The Go node reproduces the Python prototype's hashes byte for byte; a test suite checks it against vectors generated by the Python code. It also enforces the stricter validation in section 5.2 and the timestamp rules in section 3.4, which the prototype lacked.

8.3 Testnet-4

Testnet-4 went live on 10 October 2026 with genesis hash 4af17f69…bdd4b and introduced the block-hash beacon described in section 4. It runs with the relaxed 1 MB Argon2 setting. Its predecessor, testnet-3, ran from 4 to 10 October 2026 and reached block 15,975 before it was retired and archived. A single seed node at seed.entropychain.io:9001 bootstraps new peers and mines on one CPU thread to fund the faucet. Testnet ENTRO has no value, and the chain may be reset.

9. Security considerations

The protocol resists forgery, double-spending by a minority and bias by non-miners; its main open weaknesses are the small size of the testnet and its single seed node.

Threat	Status in testnet-4	Mitigation
Forged transactions	Defended	ECDSA signatures; the public key must hash to the sender's address
Double-spend within a block	Defended	Transactions validated in sequence against updated state
Transaction replay	Defended	Any transaction already on chain is rejected
Seed bias by non-miners	Defended	Seeds are unknown until their block is published
Beacon bias by the block's miner	**Mitigated**	Each attempt forfeits a block reward (section 4.3); draw windows multiply the cost
Trait grinding by NFT minters	**Defended**	Traits come from a block mined after the mint
Majority-hashrate attack	Inherent to proof of work	Grows harder as honest hashrate grows; cheap on a young testnet
Private-chain reorganization	Partly mitigated	Miners wait for sync; a stronger isolated miner can still reorganize
Timestamp manipulation	Defended	Median-time-past and 2-hour future limits
Transaction ordering by miners	Accepted	Fee-priority mempool; no fair-ordering protocol
Mempool flooding	Limited	10,000-transaction cap with fee-based eviction
Peer-to-peer denial of service	Limited	Message, connection and misbehaviour limits (section 8.1)
Seed-node outage	**Open**	One seed node today; more are planned

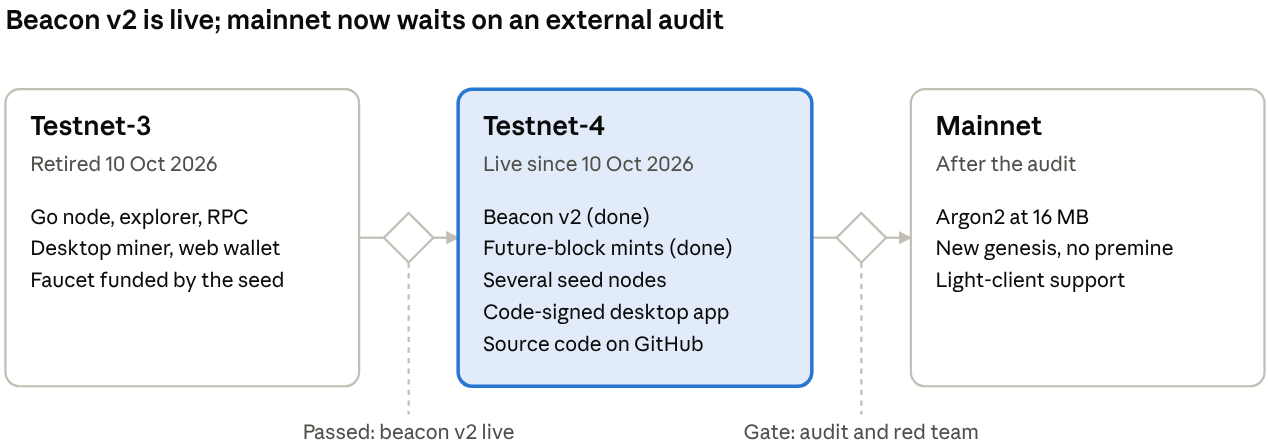
A reorganization in the first days of testnet-3 showed the private-chain risk in practice. A desktop miner that had not yet connected mined 45 blocks on its own, then reconnected with more cumulative work. The network correctly switched to that chain, discarding 85

blocks. Miners now wait until they are synced, but heaviest-chain consensus still lets any party with more hashrate replace recent history.

The security of a proof-of-work chain ultimately depends on its total honest hashrate. On a young network with few miners, an attacker can rent or bring enough CPU power to outpace everyone else, so testnet results should not be read as mainnet guarantees.

10. Roadmap

The block-hash beacon went live with testnet-4 on 10 October 2026; mainnet now waits on an independent review of the code and network.



Proposed roadmap · 3 phases, 2 gates, no dates set

Testnet-4 launched as a fresh chain because the beacon change altered consensus. Mainnet will start from a new genesis at the production Argon2 setting. No dates are set; the remaining testnet-4 work and the audit gate must be complete first.

Disclaimer

This paper describes software under active development. Testnet ENTRO has no monetary value, testnets may be reset without notice, and the protocol will change before any mainnet, including the consensus changes described above. Nothing here is an offer of securities, investment advice, or a promise of future value or functionality.

References

1. S. Nakamoto. Bitcoin: A Peer-to-Peer Electronic Cash System. 2008.

2. A. Biryukov, D. Dinu and D. Khovratovich. Argon2: New Generation of Memory-Hard Functions for Password Hashing and Other Applications. IEEE European Symposium on Security and Privacy, 2016.
3. RFC 9106: Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications. IETF, 2021.
4. RFC 6979: Deterministic Usage of the Digital Signature Algorithm (DSA) and Elliptic Curve Digital Signature Algorithm (ECDSA). IETF, 2013.
5. Certicom Research. SEC 2: Recommended Elliptic Curve Domain Parameters (secp256k1). 2010.
6. J. Bonneau, J. Clark and S. Goldfeder. On Bitcoin as a Public Randomness Source. IACR Cryptology ePrint Archive, 2015.
7. D. Boneh, J. Bonneau, B. Bünz and B. Fisch. Verifiable Delay Functions. CRYPTO 2018.

Protocol constants and behaviour in this paper are taken from the EntropyChain reference implementation as deployed on testnet-4.